



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Technical presentation

SESSION TRACKING TECHNIQUES

Guided By:

Dr. SUDHAGAR D

Presented By:

POOJASREE B



INTRODUCTION TO SESSION TRACKING



HTTP is a **stateless protocol**. Servers do not remember previous requests automatically - session tracking fills that gap by preserving user information across requests.

Why it matters

Maintains login status, shopping cart contents, user preferences, and application state across multiple page requests.

Common techniques

1. HttpSession (server-side)
2. Cookies (client-side)
3. URL Rewriting (token in URL)





HTTPSESSION (SERVER-SIDE)



HttpSession creates a unique server-side session per user. The server stores attributes keyed by a session ID that's sent to the client (usually via cookie).



Core behavior

Session data lives on the server and persists until timeout or explicit invalidation. The session ID maps requests to stored data.

Simple Java example

```
HttpSession session = request.getSession();  
session.setAttribute("username", "Divya");  
String user = (String) session.getAttribute("username");
```

Common uses

- Login authentication
- Shopping carts
- Temporary user preferences



COOKIES (CLIENT-SIDE)

Cookies are small pieces of data stored by the browser and sent with requests to the server. They can hold session IDs or lightweight user settings.

Types of cookies

Session, Persistent, Secure, HttpOnly - each has different lifespan and security properties.

Example (Java)

```
Cookie c = new  
Cookie("username", "Divya");  
response.addCookie(c);
```

Best used for

Remember-me, non-sensitive preferences, lightweight session identifiers, UI personalization.



URL REWRITING



URL rewriting injects the session ID into links when cookies are unavailable. It reliably carries session state but exposes the ID in the URL.

How it works

1. User requests a page
2. Server creates a session ID
3. ID appended to URLs (e.g., /home;jsessionid=ABC123)
4. User follows rewritten links; server reads ID to restore session

When to use

Fallback when cookies are disabled — useful for older browsers or restrictive environments.





ADVANTAGES & LIMITATIONS



HttpSession

Secure server-side storage; easy server control.

Limitation: consumes server memory and requires scaling strategies.

Cookies

Simple and persistent; widely supported.

Limitation: can be disabled, deleted, or stolen if not secured.

URL Rewriting

Works without cookies.

Limitation: session ID is visible in URLs, logs, and browser history.



No single technique is perfect — choose based on security, persistence, and application constraints.



SESSION PERSISTENCE: BEHAVIOR BY TECHNIQUE



HttpSession

Data stored server-side;
expires on timeout or logout.
Requires sticky sessions or
distributed store in clustered
environments.

Cookies

Persistent cookies survive
browser restarts if
configured; expiration
controlled by cookie
attributes.

URL Rewriting keeps state via URLs; ensure every link includes the session token or the session will be lost.





SECURITY CONCERNS & MITIGATIONS



Major risks

- ❑ Session hijacking — attacker steals a valid session ID
- ❑ Session fixation — attacker forces a known session ID
- ❑ Cookie theft via XSS or network interception
- ❑ URL exposure — IDs in logs or history

Practical defenses

- ❑ Enforce HTTPS site-wide
- ❑ Set cookies Secure and HttpOnly
- ❑ Regenerate session ID after authentication
- ❑ Use strict session timeouts and invalidate on logout
- ❑ Avoid sensitive data in URLs



USE-CASE COMPARISON & RECOMMENDATION



Best fits

- **HttpSession:** Java web apps, complex server-side state, shopping carts
- **Cookies:** Remember-me, user preferences, light session IDs
- **URL Rewriting:** Environments where cookies are disabled

Recommended modern approach

Combine secure, HttpOnly cookies for session identifiers with server-side sessions (HttpSession or distributed store). This balances usability, persistence, and security while minimizing URL exposure.



For scalable apps, use distributed session stores (Redis, database) or stateless tokens with careful security controls.



CONCLUSION



Session tracking enables stateful user experiences on the stateless web. Choose between server-side sessions, cookies, and URL rewriting based on security needs, persistence requirements, and client capabilities.

Key decisions

- ☐ Prioritize HTTPS
- ☐ Regenerate IDs on login
- ☐ Prefer Secure + Http Only cookies
- ☐ Use URL rewriting only as fallback.